专为AI设计的Python课程

1. 本套课程的说明
2. 基本类型与基本运算
   2.1. 认识变量类型和变量地址
   2.2. 余数 (%) 和整除(//)
   2.3. 乘方(**)
   2.4. 位运算操作
   2.5. 整数与浮点数的特性
   2.6. 字符串
   2.7. 输入与输出
3. 容器类型与对应操作行为
   3.1. 列表
   3.2. 元组类型
   3.3. 字典类型
   3.4. 集合
4. 分支结构
   4.1. 单分支
   4.2. 双分支
   4.3. 多分支
   4.4. match匹配语句
5. 循环结构
   5.1. for循环
   5.2. while循环
6. 函数定义
7. 面向对象
8. 文件操作
9. 异常处理
   9.1. 一个除数为 0 的错误
   9.2. 撰写异常处理程序 try - except - else - finally

9.3. 自定义异常与丢出异常
10. 模块化编程
10.1. math 模块
10.2. random 模块
10.3. 日期时间模块
10.4. 自定义模块
10.5. 第三方模块
10.5.1. 网络爬虫
10.5.2. 自动化
10.5.3. 数据分析及可视化
10.5.4. WEB开发
10.5.5. 机器学习
10.5.6. 其他常用

1. 本套课程的说明

后续系列中, 我将为大家讲解如下的语法内容：

1. Python 语言的基本数据类型和基本运算操作
2. Python 语言中的容器类型与对应操作行为
3. Python 语言中的分支结构
4. Python 语言中的循环结构
5. Python 语言中的函数定义
6. Python 语言中的面向对象特性与语法实现
7. Python 语言的文件操作
8. Python 语言的异常处理机制
9. Python 语言的模块管理

2. 基本类型与基本运算

2.1. 认识变量类型和变量地址

- Python可以使用 id() 函数获得变量的地址.
- 相同内容的变量会有相同的地址.

id() 内置函数查看地址 Python

```python
# 数据类型 变量名 = 变量值
# Python 语言的变量类型是自动推导

# 1. 整型常量特点
num1 = 10
num2 = 10

# 思考 : 以上两个变量是一样的吗?
# %s 字符串    %d  整型    %f 浮点型       字符串.format()
# f-string
# print(f'id(num1) = {id(num1)}, id(num2) = {id(num2)}')
print(f'{type(num1) = }, {type(num2) = }')
print(f'{id(num1) = }, {id(num2) = }')

print('-' * 50)      # 字符串相乘就是重复的含义

# 2. 浮点型常量特点
n1 = 10.5
n2 = 10.5
print(f'{type(n1) = }, {type(n2) = }')
print(f'{id(n1) = }, {id(n2) = }')

print('-' * 50)

# 3. 字符串常量的特点
s1 = 'hello world'       # Python 语言的字符串都是一样的. 没有所谓 `字符类型` char
s2 = "hello world"
print(f'{type(s1) = }, {type(s2) = }')
print(f'{id(s1) = }, {id(s2) = }')
```

输出结果 Python

```
type(num1) = <class 'int'>, type(num2) = <class 'int'>
id(num1) = 140711479149768, id(num2) = 140711479149768
--------------------------------------------------
type(n1) = <class 'float'>, type(n2) = <class 'float'>
id(n1) = 2073531519696, id(n2) = 2073531519696
--------------------------------------------------
type(s1) = <class 'str'>, type(s2) = <class 'str'>
id(s1) = 2073531578352, id(s2) = 2073531578352
```

2.2. 余数 (%) 和整除(//)

使用 divmod() 函数一次获得商和余数

divmod(x,y)	商余，(x//y, x%y)，同时输出商和余数 divmod(10, 3) 结果为 (3, 1)

divmod() 函数 Python

```python
# 商 (没有小数点)
quotient1 = 10 / 3     # / 浮点除
quotient2 = 10 // 3     # // 整除
print(f'{quotient1 = }, {quotient2 = }')

# 余数
remaining = 10 % 3
print(f'{remaining = }')

# 内置函数 divmod(x, y)
# 逗号表达式
q, r = divmod(10, 3)        # 商, 余数
print(f'{q = }, {r = }')
```

输出结果 Python

```
quotient1 = 3.3333333333333335, quotient2 = 3
remaining = 1
q = 3, r = 1
```

2.3. 乘方(**)

乘方 Python

```python
# 二进制
# 复制  ctrl + d (duplicate 复制)
print(2 ** 0)       # 1
print(2 ** 1)
print(2 ** 2)
print(2 ** 3)
print(2 ** 4)
print(2 ** 5)
print(2 ** 6)
print(2 ** 7)

print('-' * 20)

print(10 ** 0)
print(10 ** 1)
print(10 ** 2)
```

思考：普通人如何学习复利思维?

Python

```python
# 思考 : 普通人如何学习复利思维?
# 点 -> 线 -> 面 -> 体 -> 灵活使用

# 1. 一年中每一天都进步 0.1% (千分之一), 请问一年之后, 你进步了多少?
day_up = 1.001 ** 365
print(f'{day_up:.2f}')

# 2. 一年中每一天都退步 0.1% (千分之一), 请问一年之后, 你还剩余多少?
day_down = 0.999 ** 365
print(f'{day_down:.2f}')
```

输出结果 Python

```
1.44
0.69
```

2.4. 位运算操作

需求：交换 x, y 的值

交换两个变量的数值 Python

```python
# 需求 : 交换 x, y 的值
x = 10
y = 20

# 方式一 : 错误
# x = y
# y = x

# 方式二 : 临时变量
# temp = x
# x = y
# y = temp

# 方式三 : + - -    (自增运算符 ++    自减运算符 --)
# x = x + y       # x = 30
# y = x - y       # y = 10
# x = x - y       # x = 20

# 方式四 : Python 语言特有的交换语法, 其它语言没有该语法
# x, y = y, x

# 方式五 : 位运算 - 异或操作 (口诀: 相同为0, 不同为1)
'''          一个字节表示数据
    10       0000 1010
    20       0001 0100
'''

'''          一个字节表示数据
    10       0000 1010
    20       0001 0100
    -----------------
             0001 1110       x=30
'''
x = x ^ y

'''          一个字节表示数据
    30       0001 1110
    20       0001 0100
    -----------------
             0000 1010       y=10
'''
y = x ^ y

'''          一个字节表示数据
    30       0001 1110
```

```
    10      0000 1010
    ----------------
            0001 0100       x=20
'''
x = x ^ y

print(f'{x = }, {y = }')
```

输出结果 Python

```
x = 20, y = 10
```

其它位运算法操作：

Python

```python
# << 左移  : 每移动一位, 相当于 * 2
n = 10
n = n << 1
print(f'{n = }')

# >> 右移  : 每移动一位, 相当于 // 2
m = 10
m = m >> 1
print(f'{m = }')

# & 按位与  : 全 1 为 1
a = 10
b = 7
'''
    10      0000 1010
    7       0000 0111
    ------------------
            0000 0010     结果为: 2

'''
c = a & b
print(f'{c = }')

# | 按位或  : 有 1 就 1
'''
    10      0000 1010
    7       0000 0111
    ------------------
            0000 1111    结果为: 15
'''
d = a | b
print(f'{d = }')
```

Python

```python
n = 20
m = 5
c = 2
d = 15
```

2.5. 整数与浮点数的特性

整数的英文是 integer, 在计算机程序语言中一般用 int 表示. 整数大小必须在某一区间, 否则会有溢位(overflow), 造成数据不正确. Python3已经将整数存储空间的大小限制拿掉了, 所有没有 long 了, 也就是说 int 可以是任意大小的数值.

浮点数的英文是 float, 既然整数大小没有限制, 浮点数大小当然也没有限制.

Python

```python
# 整型与浮点型都是有数值限定的
'''
    int  整型, 占用 4 个字节空间, 总有 4 * 8 = 32 个比特位.   31个比特位可以存储数据.  正数 2147483648  负值 -2147483647
    unsigned int, unsigned long 无符号类型, 只能存储正数, 不能存储负数. 4294967296, 18446744073709551616
    long 长整型, 占用 8 个字节空间, 最大值为 9223372036854775808
    float 单精度浮点型 4个字节,   double 双精度浮点型 8 个字节
'''

# 科学计算 : 航天领域, 宇宙学, 天文学 ...
n = 18446744073709551616 + 1
print(n)

f = 123456789.123456789
print(f)
```

输出结果 Python

```
18446744073709551617
123456789.12345679
```

2.6. 字符串

所谓的字符串(string)数据是指两个单引号之间或是两个双引号之间任意个数字元符号的数据, 它的数据类型代号是 str, 如果我们用单引号去处理字符串产生错误, 碰到这种情况, 我们可以用双引号解决.

- 数学的运算符 + 可以执行两个字符串相加, 产生新的字符串.
- 程序设计时如果字符串长度多于一行, 可以使用三个单引号(或是三个双引号)将字符串包夹.

字符串数据类型 Python

```python
# 字符串 : 字符 + 字符串
# 1. 单引号   ''
# 2. 双引号   ""
# 3. 三引号   '''   '''    """  """    多行字符串


s = '''
        介绍信
  尊敬的xxx, 我向您推荐该学员...
  ...
  ...
  非常优秀!!
                    此致/敬礼
'''
print(s)
```

Python

```python
# raw string 原生字符串 (取消所有转义)
# 文件路径
filepath = 'D:\\TuLing\\2421\\视频\\专为AI设计的Python课程'
filename = r'D:\TuLing\2421\视频\专为AI设计的Python课程'
print(f'{filepath = }')
print(f'{filename = }')


# binary string
# 二进制数据
s = b'hello world'      # 相当于将字符串类型数据转换为了字节数据
print(s, type(s))
```

输出结果 Python

```
filepath = 'D:\\TuLing\\2421\\视频\\专为AI设计的Python课程'
filename = 'D:\\TuLing\\2421\\视频\\专为AI设计的Python课程'
b'hello world' <class 'bytes'>
```

2.7. 输入与输出

input()函数与print()函数功能相反, 这个函数会从屏幕读取用户从键盘输入的数据. 特别注意所有输入的数据不论是字符串或是数值数据返回到value时一律是字符串数据, 如果要执行数学运算需要用

int()函数转换为整数.

Python

```
1
```

输出结果 Python

```
1
```

eval() 函数： evaluate

eval(<字符串>)函数是Python语言中一个十分重要的函数，它能够以Python表达式的方式**解析并执行字符串**，将返回结果输出。

思考：输入多个数字, 为什么只能用逗号分隔? 为什么使用空格分隔会报错呢?

eval() 函数 Python

```python
# public static void main() {
#     system.out.print("hello world")
# }


print('hello world')

# input() 函数是一个阻塞函数, 回车才代表输入的结束
msg = input('请输入你的内容: ')
print(f'{msg = }, {type(msg) = }')

# 需求 : 让用户输入两个整型数值, 实现数学上的加法运算
n1, n2 = eval(input('请输入两个整型数值: '))     # 10, 20
result = n1 + n2
print(f'{result = }')
```

输出结果 Python

```
hello world
请输入你的内容: 10
msg = '10', type(msg) = <class 'str'>
请输入两个整型数值: 10, 20
result = 30
```

3. 容器类型与对应操作行为

3.1. 列表

Python

```python
# 定义一个列表对象
# alist = []
alist = list()
print(f'{alist = }, {type(alist) = }')

# CRUD 增删改查
alist.append(1)
alist.append(2)
alist.append(3)
print(alist)

alist.insert(0, 9)
print(alist)

element = alist.pop()      # 弹出
print(alist, element)

# 指定下标弹出元素
element = alist.pop(0)
print(alist, element)

# 指定元素删除
alist.remove(2)
print(alist)

# 修改
alist[0] = 666
print(alist)
```

输出结果 Python

```
alist = [], type(alist) = <class 'list'>
[1, 2, 3]
[9, 1, 2, 3]
[9, 1, 2] 3
[1, 2] 9
[1]
[666]
```

```
mylist[start:end:step]    # 每隔 step，读取从索引 start 到 end-1 的列表元素
```

列表索引值是 -1, 代表是最后一个列表元素. -2代表最后2个元素, 其它负索引概念可以此类推.

列表切片 Python

```python
# 切片的语法（容器类型 / 序列类型）
# [start : stop : step]   下标 : start=0, stop=10（不包含）   step = 1
alist = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# blist = alist[0:5:1]
blist = alist[:5:]
print(blist)

str = 'hello world'
substr = str[0:5:1]
print(substr)
```

输出结果 Python

```
[1, 2, 3, 4, 5]
hello
```

列表.sort() 与 sorted() 区别 Python

```python
# 存取有序
alist = list([5, 7, 3, 2, 8, 9, 1, 4, 6])
print(alist)

# 元素有序（升序）
alist.sort()
print(alist)

alist.reverse()     # 降序
print(alist)

print('-' * 20)

blist = list([5, 7, 3, 2, 8, 9, 1, 4, 6])
blist.sort(reverse=True)
print(blist)

print('-' * 20)

# 内置函数 sorted() 不是在原列表上进行排序的，而是复制了一个新排列再实现排序. 所以操作完毕后一定要接收返回的结果
clist = list([5, 7, 3, 2, 8, 9, 1, 4, 6])
dlist = sorted(clist, reverse=True)
print(clist, id(clist))
print(dlist, id(dlist))
```

输出结果 Python

```
[5, 7, 3, 2, 8, 9, 1, 4, 6]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
[9, 8, 7, 6, 5, 4, 3, 2, 1]
--------------------
[9, 8, 7, 6, 5, 4, 3, 2, 1]
--------------------
[5, 7, 3, 2, 8, 9, 1, 4, 6] 1317187550976
[9, 8, 7, 6, 5, 4, 3, 2, 1] 1316895498560
```

3.2. 元组类型

元组的功能

1. 可以更安全地保护数据

程序设计中有些数据可能永远不会改变, 将它存储在元组(tuple)内, 可以被安全地保护, 例如: 图像处理时对象的长,宽或每一像素的色彩数据, 很多是元组数据类型.

2. 加快程序执行速度

元组 (tuple) 结构比列表 (list) 简单, 占用较少的系统资源, 程序执行时速度比较快.

了解了上述元组的优点后, 未来设计程序时, 如果确定数据可以不更改, 就尽量使用元组数据类型

Python

```python
# 定义一个元组类型
# tuple 元素不可变

tuple1 = (1, 2, 3, 4, 5)
print(tuple1, type(tuple1))

# tuple1[0] = 6
# print(tuple1, type(tuple1))

# 细节 (元组中仅有单个元素, 需要在元素之后添加逗号, 不能省略)
tuple2 = (1, )
print(tuple2, type(tuple2))
```

输出结果 Python

```
(1, 2, 3, 4, 5) <class 'tuple'>
(1,) <class 'tuple'>
```

3.3. 字典类型

keys()与values()与items()

```python
'''
    字典类型 : key-value 键值对, 通过 key 实现操作, key 要保证唯一性

    Java : Map 接口   HashMap / LinkedHashMap   哈希表
    Python : dict()   {key: value}
'''

adict = {'张三': 100, '李四': 98, '王五': 88, '赵六': 60}
print(adict)

adict['张三'] = 90
print(adict)

score = adict['张三']
print(f'{score = }')

# keys 键集, values 值集, items 键值对集合
print(adict.keys())
print(adict.values())
print(adict.items())
```

输出结果

```
{'张三': 100, '李四': 98, '王五': 88, '赵六': 60}
{'张三': 90, '李四': 98, '王五': 88, '赵六': 60}
score = 90
dict_keys(['张三', '李四', '王五', '赵六'])
dict_values([90, 98, 88, 60])
dict_items([('张三', 90), ('李四', 98), ('王五', 88), ('赵六', 60)])
```

大型程序设计中, 字典用久了会产生相当多的元素, 也许是几千个或几十万个或更多.

3.4. 集合

(一) 确定性

(二) 无序性

(三) 互异性

```python
# 集合  set() / {e1, e2, e3 ...}

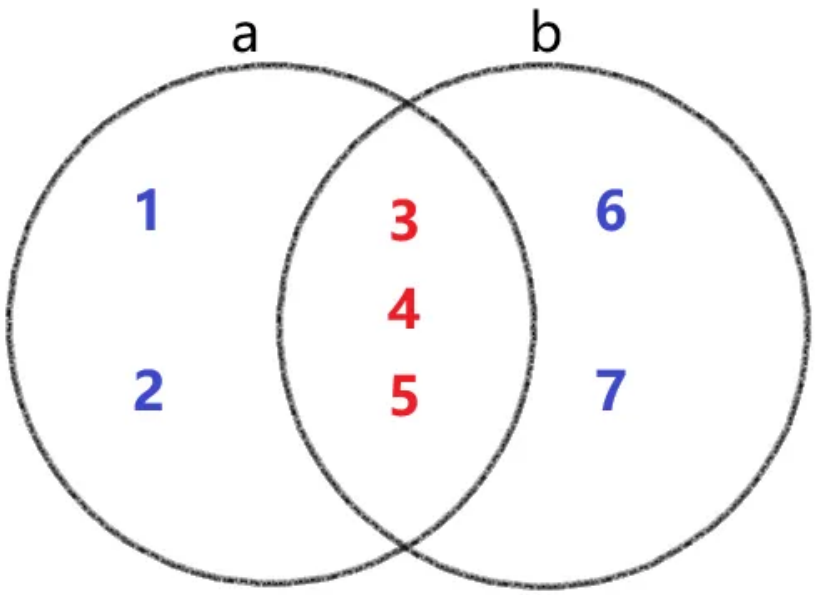
aset = {11, 22, 33, 44, 55, 44, 33, 22, 11}
print(aset)
```

输出结果

```python
{33, 22, 55, 11, 44}
```

操作说明与Venn图

```
a = {1, 2, 3, 4, 5}
b = {3, 4, 5, 6, 7}
```



Python 符号	说明
&	交集
|	并集
-	差集
^	对称差集
==	等于
!=	不等于
in	是成员
not in	不是成员

交集 (intersection &)

并集 (union |)

差集 (difference -)

对称差集 (symmetric difference)

等于 == 与不等于 !=

元素属于集合, Python 的关键词 in 可以测试元素是否属于集合

元素不属于集合, Python的关键词 not in 可以测试元素是否不属于集合

综合练习 Python

```python
a = {1, 2, 3, 4, 5}
b = {3, 4, 5, 6, 7}

# 交集
# r1 = a & b
r1 = a.intersection(b)
print(r1)

# 并集
# r2 = a | b
r2 = a.union(b)
print(r2)

# 差集
r3 = a - b
r4 = b - a
print(r3)
print(r4)

# 对称差集
r5 = a.symmetric_difference(b)
print(r5)
```

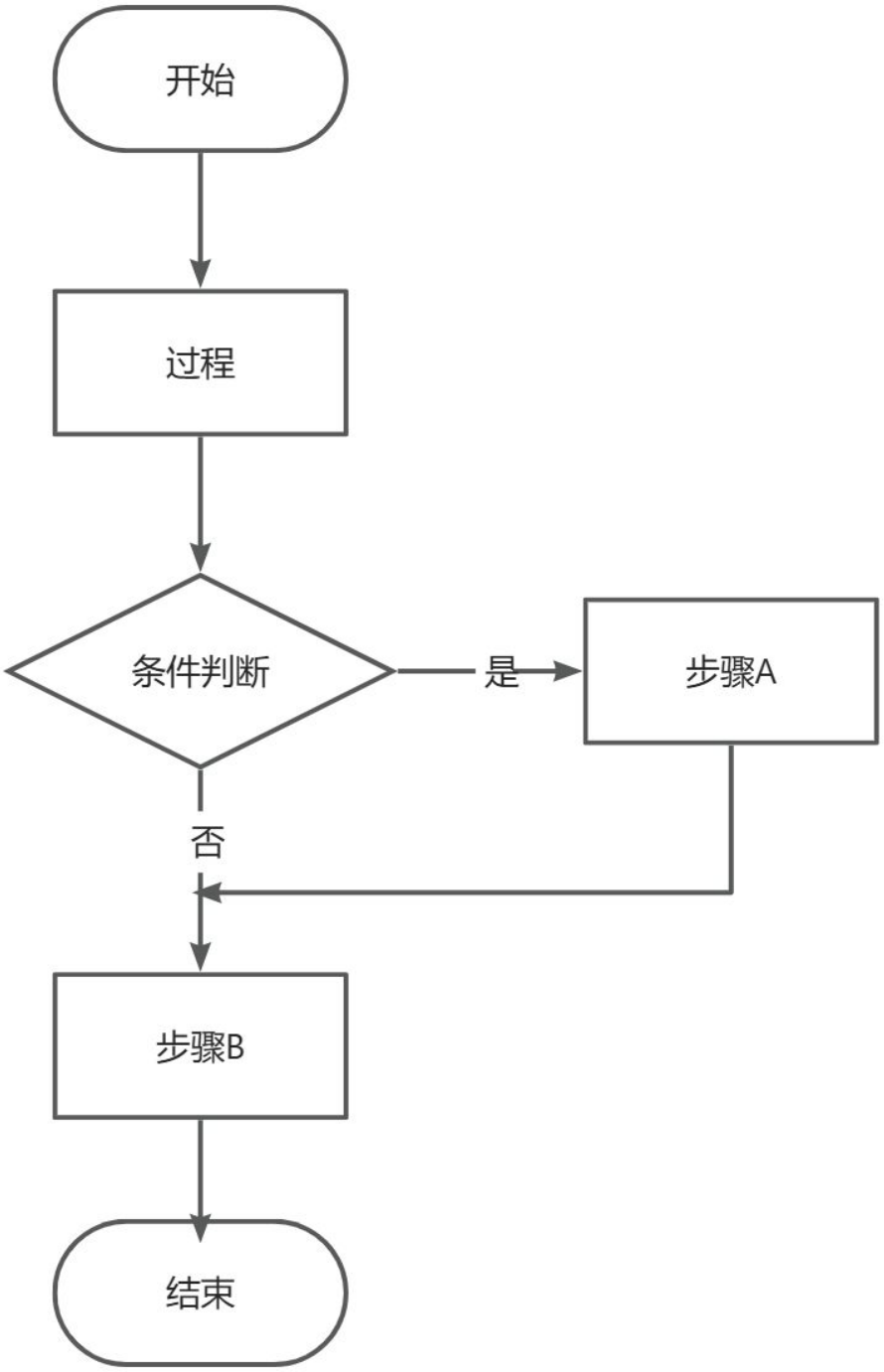
输出结果 Python

```
{3, 4, 5}
{1, 2, 3, 4, 5, 6, 7}
{1, 2}
{6, 7}
{1, 2, 6, 7}
```

4. 分支结构

4.1. 单分支

单分支结构




```
if 条件:
    语句1
    语句2
    语句n
步骤B
```

1. 条件判断外的括号可有可无.
2. 在 Python 内使用缩进方式区隔 if 语句的程序代码区块. 编辑程序时可以用 Tab 键缩进或是直接缩进 4 个字符空间, 表示这是 if 语句的程序代码区块.
3. 在Python中缩进程序代码是有意义的, 相同的程序代码区块, 必须有相同的缩进, 否则会产生错误.

课堂案例：小明的语文, 数学, 英语成绩, 判断三门功课是否全部及格.

课堂案例 Python

```python
# 接收三科成绩
chinese, math, english = eval(input('请输入小明的语文,数学,英语成绩: '))

# 判断
if chinese >= 60 and math >= 60 and english >= 60:
    print('全部及格!')

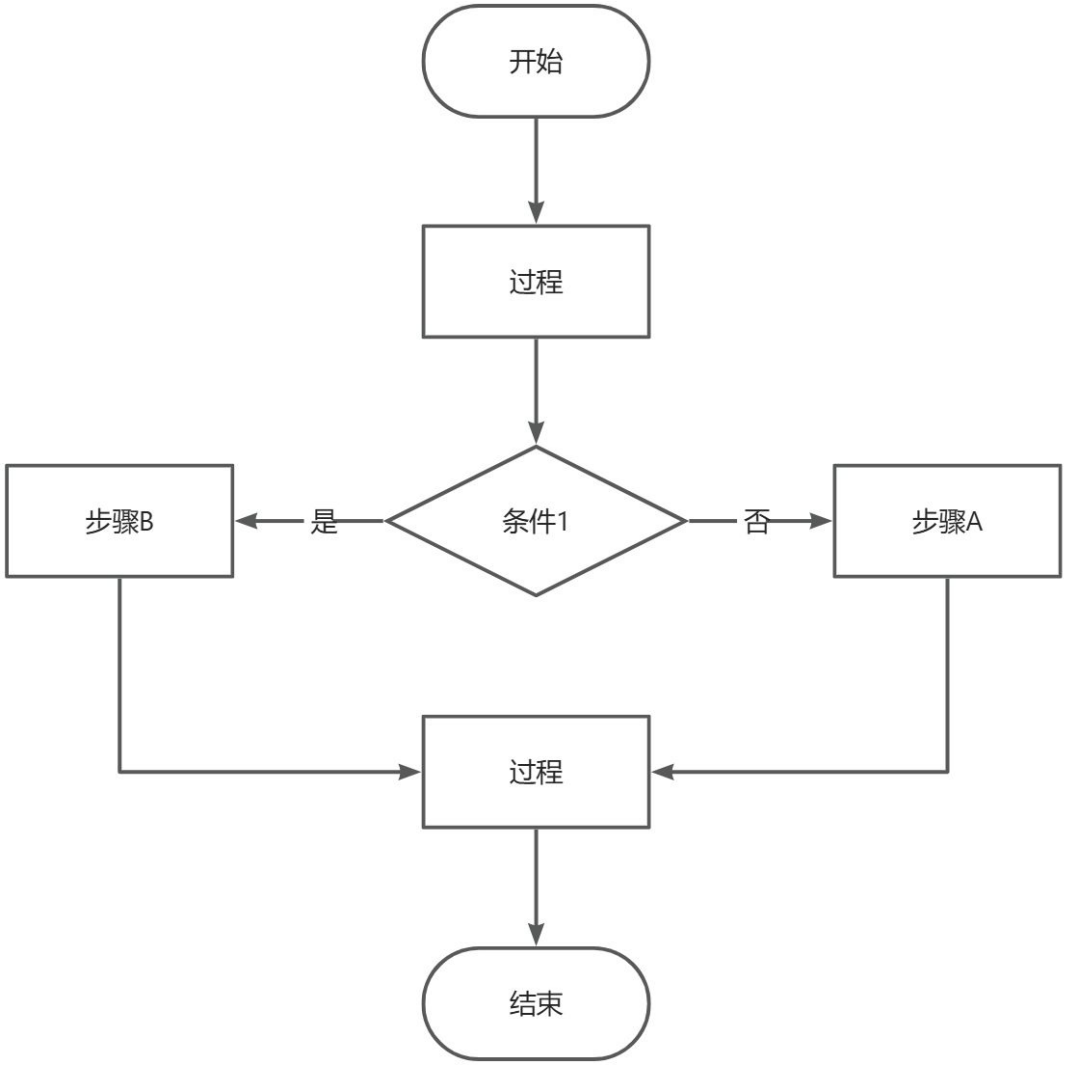
print('程序继续向下执行...')
```

输出结果 Python

```
请输入小明的语文,数学,英语成绩: 60, 60, 59
程序继续向下执行...
```

4.2. 双分支

双分支结构




```
if 条件:
    语句1
    语句2
    语句n
else:
    语句1
    语句2
    语句n
过程
```

实现奇偶数的判断

实现奇偶数的判断 Python

```python
# 实现奇偶数的判断
n = int(input('请输入一个整型数值: '))

# 判断
if n % 2 == 0:
    # 偶数
    print(f'{n}是一个偶数.')
else:
    # 奇数
    print(f'{n}是一个奇数.')
```

输出结果 Python

```
请输入一个整型数值: 8
8是一个偶数.
```

双分支简化

输出较大值 Python

```python
# 获取两个整型数值的较大值
n1, n2 = eval(input('请输入两个整型数值: '))

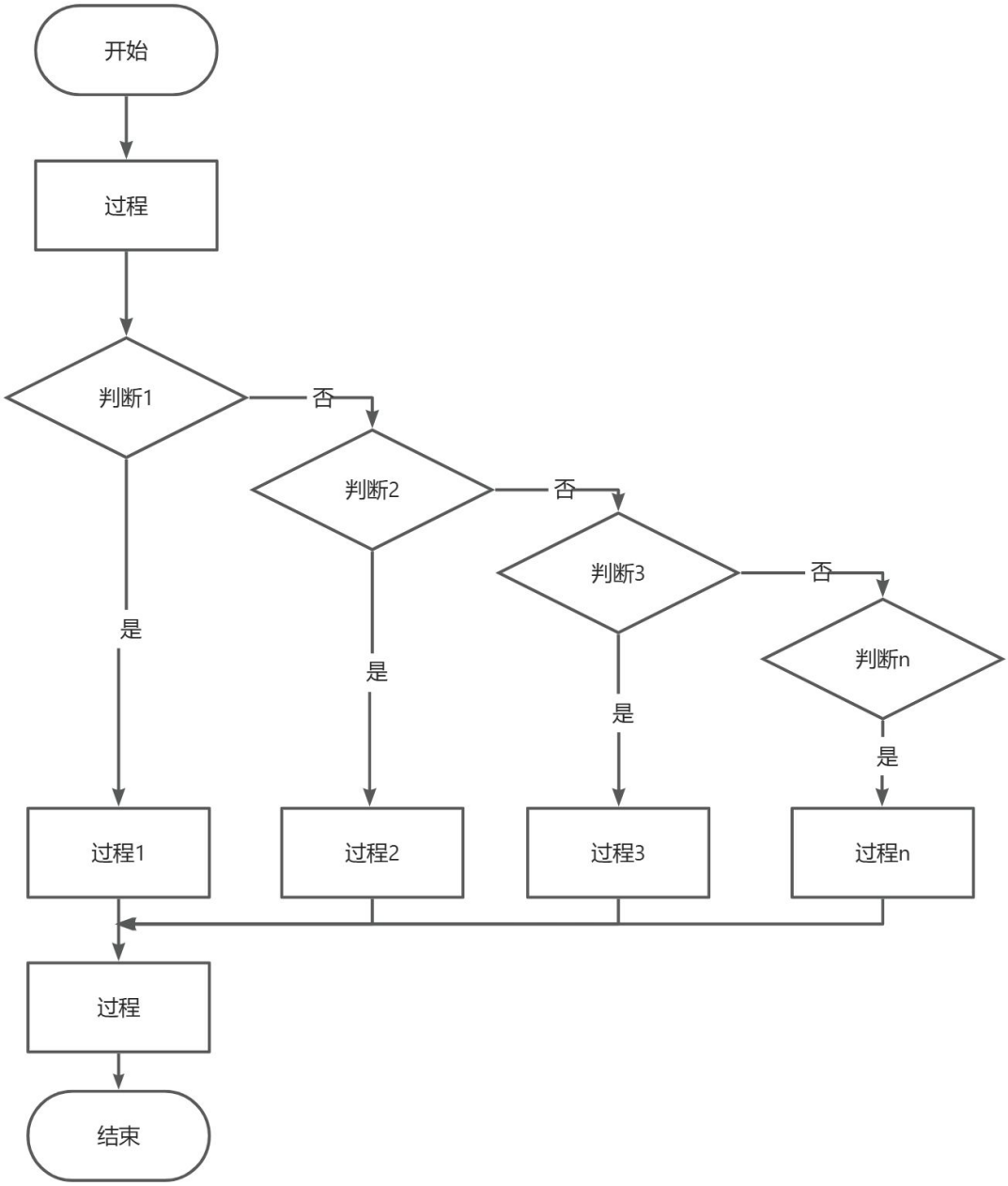
# 判断
bigger = n1 if n1 > n2 else n2
print(f'{bigger = }')
```

输出结果 Python

```
请输入两个整型数值: 12, 11
bigger = 12
```

4.3. 多分支

多分支结构




```
if 条件1:
    语句1
    语句n
elif 条件2:
    语句1
    语句n
elif 条件3:
    语句1
    语句n
elif 条件n:
    语句1
    语句n
过程
```

- 练习 1 : 请输入数字分数, 系统将响应A, B, C, D 或 F 等级.

计算最终成绩等级 Python

```python
# 请输入数字分数, 系统将响应A, B, C, D 或 F 等级.
score = eval(input("Enter score: "))

# 多分支判断
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("F")
```

Python

```
Enter score: 59
F
```

4.4. match匹配语句

```python
# 请输入数字分数, 系统将响应A, B, C, D 或 F 等级.
score = eval(input("Enter score: "))

# 多分支判断
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
elif score >= 70:
    print("C")
elif score >= 60:
    print("D")
else:
    print("F")

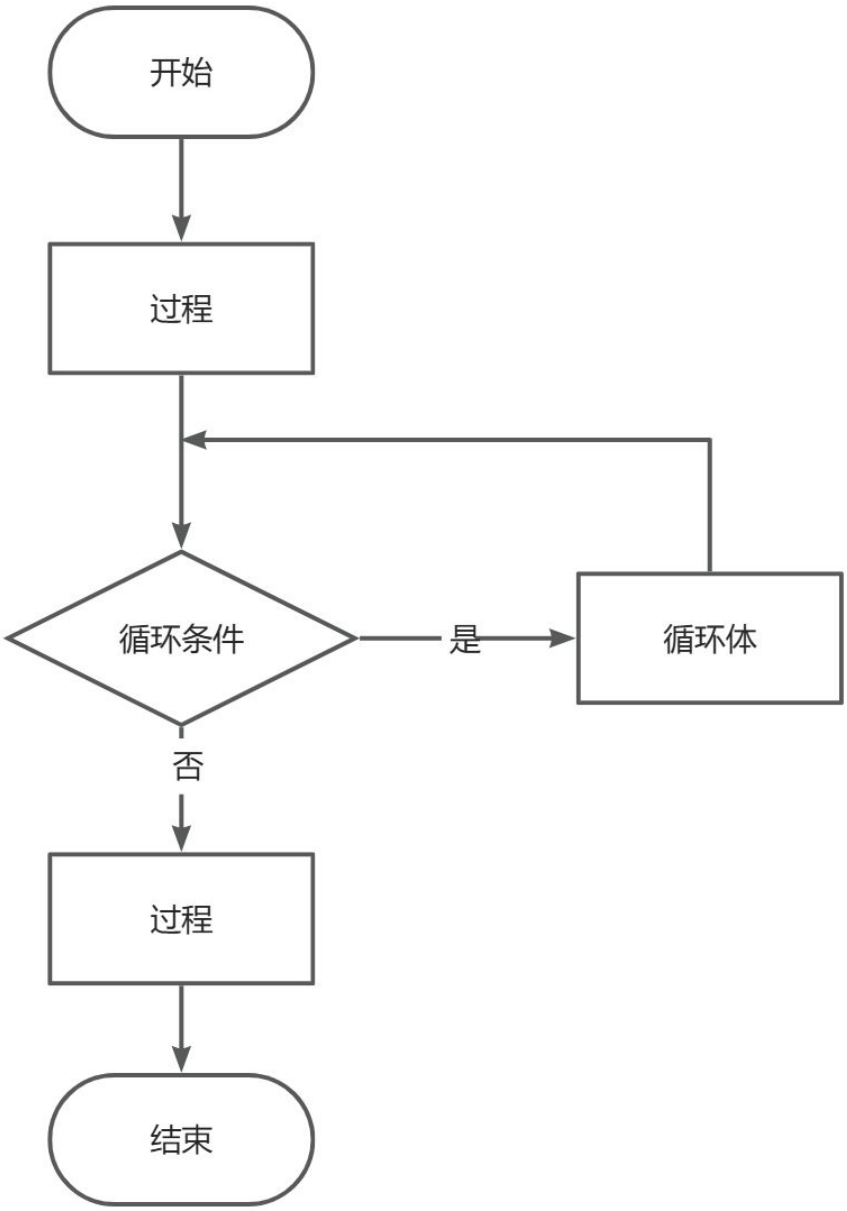
# match 匹配语句, 类似于其它语言的 switch 语言
level = score // 10
match level:
    case 10 | 9:
        print("A")
    case 8:
        print("B")
    case 7:
        print("C")
    case 6:
        print("D")
    case _:     # 兜底条件
        print("F")
```

```
Enter score: 77
C
C
```

5. 循环结构

5.1. for循环

可迭代对象 (iterable object) 可以是列表, 元组, 字典, 集合或 range(), 在信息科学中迭代 (iteration) 可以解释为重复执行语句.




```
for 循环条件
    循环体语句1
    循环体语句2
    循环体语句3
    循环体语句n
过程
```

```
range(start, end, step)  #  start 是起始值，end-1 是终止值，step 是间隔值
```

range()函数 Python

```python
# for 循环 : 执行指定次数的循环

# 需求 : 我爱北京天安门
# range() 必须要指定 stop 参数数值
for _ in range(10):     # [0~9] 序列中的下标
    print('我爱北京天安门')

# 循环使用场景 : 序列
alist = [11, 22, 33, 44, 55]
for i in alist:
    print(i)

# 字符串序列
str = 'hello world!'
for i in str:
    print(i)
```

生成式 (generator) 是一种使用迭代方式产生 Python 数据的方式, 可以产生列表, 字典, 集合等. 这是结合循环与条件表达式精简程序代码的方法, 如果会使用此概念设计程序, 表示你的 Python 功力已跳出初学阶段, 如果你有其他程序语言经验, 表示你已经逐渐跳出其他程序语言的枷锁, 逐步脱变成真正的 Python 程序设计师.

列表生成式 (list generator) 的应用 Python

```python
# 需求 1 : 生成 10 个 0 的列表对象
alist = [0] * 10
print(alist)

# 需求 2 : 生成 1~10 之间的所有偶数序列
# for i in range(2, 11, 2):
#     print(i)

blist = [i for i in range(2, 11, 2)]
print(blist)

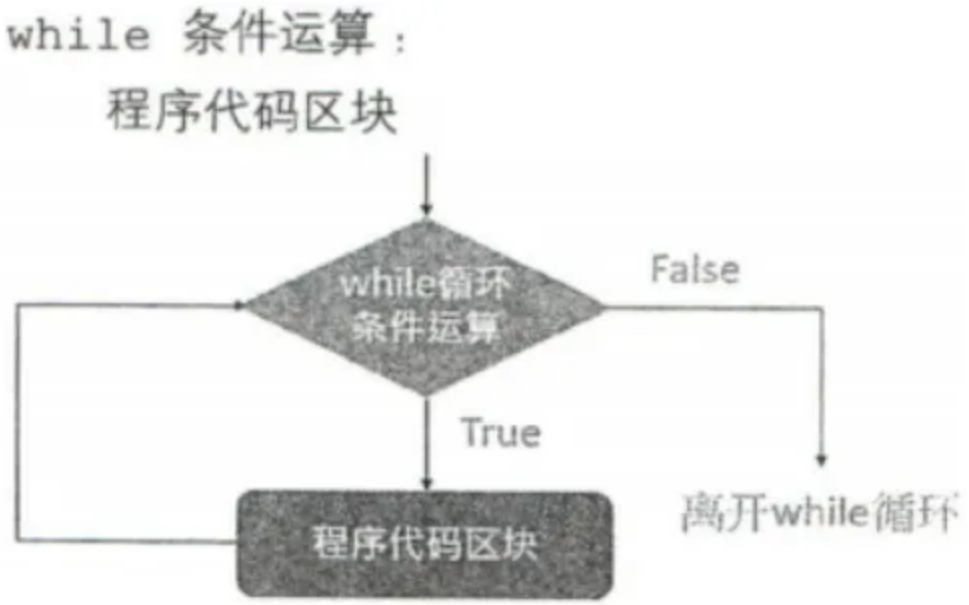
# 需求 3 : 列出 20 之内的所有勾股数   [3, 4, 5]  [5, 12, 13] ...
# 勾股定义 : a² + b² = c²
# for a in range(1, 20, 1):
#     for b in range(a, 20, 1):
#         for c in range(b, 20, 1):
#             # 判断
#             if a ** 2 + b ** 2 == c ** 2:
#                 print(a, b, c)

clist = [(a, b, c) for a in range(1, 20, 1) for b in range(a, 20, 1) for c in range(b, 20, 1) if a ** 2 + b ** 2 == c ** 2]
print(clist)
```

输出结果 Python

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
[2, 4, 6, 8, 10]
[(3, 4, 5), (5, 12, 13), (6, 8, 10), (8, 15, 17), (9, 12, 15)]
```

5.2. while循环



- 这也是一个循环, 基本上循环会一直执行, 直到条件运算为 False 才会离开循环, 所以设计 while 循环时一定要设计一个条件可以离开循环, 相当于让循环结束.
- 程序设计时, 如果忘了设计可以离开循环的条件, 程序会造成无限循环状态.
- 一般 while 循环使用的是条件控制循环, 在符合特定条件下执行, for 循环则是一种计数循环, 会重复执行特定次数.

认识哨兵值

- 在程序设计时, 我们可以在 while 循环中设定一个输入数值当做循环执行结束的值, 这个值称哨兵值 (sentinel value)

哨兵值, 用于控制 while 循环的条件 Python

```python
# 30 -> 30    30 -> 3
# 有一筐鸡蛋, 至少有两个, 两个两个数, 多一个, 三个三个数, 多一个, 四个四个数, 还是多一个, 请问, 至少有几个?


count = 2

# 死循环
while True:
    # 判断
    if count % 2 == 1 and count % 3 == 1 and count % 4 == 1:
        print(f'{count = }')
        break
    count += 1

print('-' * 20)

print('人机复读机对话!')

# 哨兵值
active = True
while active:
    dialog = input()     # 用户输入对话 (没有提示词)
    if dialog == 'quit':         # 退出
        active = False      # 将哨兵值修改为 `假`
    print(dialog)
```

输出结果 Python

```
count = 13
--------------------
人机复读机对话!
hello
hello
how old are you
how old are you
怎么老是你呀
怎么老是你呀
quit
quit
```

6. 函数定义

传递多个参数

传递多个参数 Python

```python
# 定义一个函数
# 1. 返回值类型 void, int, float, double, list, dict ...
# 2. 函数名（见名知意）
# 3. 参数列表（数据往往是需要指定类型）

# def greeting(name, age):
#     pass        # 让语法检测先通过

def greeting(name: str, age: int) -> str:
    name = name.title()     # 名字的首字母大写
    # age 修改
    age = 18
    return f'Hello {name}, you are {age} years old. you are very handsome.'


# 调用函数
msg = greeting('jack', 18)
print(f'{msg = }')
```

输出结果 Python

```
msg = 'Hello Jack, you are 18 years old. you are very handsome.'
```

回传多个数据的应用（实质是回传tuple）

回传多个数据的应用 Python

```python
# 定义函数 : 返回多个结果
def math_operation(x: int, y: int) -> tuple:
    add = x + y
    minus = x - y
    multiply = x * y
    divide = x / y
    return add, minus, multiply, divide


result = math_operation(5, 2)
print(f'{result = }, {type(result) = }')
```

输出结果 Python

```
result = (7, 3, 10, 2.5), type(result) = <class 'tuple'>
```

在设计Python的函数时, 有时候可能会碰上不知道会有多少个参数会传递到这个函数, 此时可以用下列方式设计.

不定长参数 Python

```python
# 定义函数 : 参数是不定长参数
def math_operation(*args) -> int:
    # args 本质是元组类型
    res = 0
    for arg in args:
        res += arg
    return res


r = math_operation(1,2,3,4,5)
print(f'{r = }')
```

输出结果 Python

```
r = 15
```

我们可以设计含任意数量关键词参数的函数, 方法是在函数内使用 **kwargs(kwargs是程序设计师可以自行命名的参数, 可以想成 key word argumengs) 这是关键词参数将会变成任意数量的字典元素, 其中自变量是键, 对应的值是字典的值

Python

```python
# 定义函数 : 关键字参数
def store_info(name: str, **kwargs) -> dict:
    # keyword arguments 关键字参数, 本质是一个字典类型
    """
    功能 : 存储对应姓名的学生信息
    :param name: 学生姓名
    :param kwargs: python: 编程成绩  chinese: 语文成绩  math: 数学成绩  english: 英语成绩
    :return: 返回一个封装完毕的字典数据 {name: 姓名, score: 总成绩}
    """
    name = name.title()     # 名字首字母大写
    python = kwargs['python']
    chinese = kwargs['chinese']
    math = kwargs['math']
    english = kwargs['english']
    # 将四科成绩实现累加
    score = python + chinese + math + english
    return {'name': name, 'score': score}


# 调用
r = store_info('jack', python=90, math=98, chinese=100, english=60)
print(f'{r = }')
```

输出结果 Python

```
r = {'name': 'Jack', 'score': 348}
```

匿名函数最大特色是可以有许多的参数, 但是只能有一个程序表达式, 然后可以将执行结果回传

匿名函数 lambda 的语法 Python

```python
# 匿名函数    lambda 表达式
def square(x):
    return x*x

r1 = square(5)
print(r1)

# 这个就是匿名函数
lambda x: x*x
```

filter(func, iterable) 过滤函数 Python

```python
# 过滤函数 filter(func, iterable)
alist = [i for i in range(1, 101, 1)]

# r = filter(lambda x: x % 3 == 0, alist)
r = list(filter(lambda x: x % 3 == 0, alist))

print(r)
```

Python

```
[3, 6, 9, 12, 15, 18, 21, 24, 27, 30, 33, 36, 39, 42, 45, 48, 51, 54, 57, 60, 63, 66, 69, 72, 75, 78, 81, 84, 87, 90, 93, 96, 99]
```

7. 面向对象

设计两个共性的 `动物类` 与 `食物类`

动物类子类有: 狗类,猫类,鸟类都继承自动物类. 有名称属性, 和吃的行为.

食物类子类有 : 骨头, 鱼, 虫子

最后设计一个Person类,拥有喂动物的行为.

















思考：

1. 当父类的某些方法, 需要声明, 但是又不确定如何实现时, 怎么办?
2. 不需要实例化父类对象, 父类主要作用是设计和制定规范, 让其它类来继承并实现, 怎么办?
3. 解决方案 –> 抽象类

抽象类的介绍

1. 默认情况下, Python不提供抽象类, Python附带一个模块, 该模块为定义抽象基类提供了基础, 该模块名称为 ABC
2. 当我们需要抽象基类时, 让类继承 ABC (abc模块的ABC类), 使用 @abstractmethod 声明抽象方法 (@abstractmethod 用于声明抽象方法的装饰器, 在abc模块中), 那么这个类就是抽象类
3. 抽象类的价值更多作用是**在于设计**, 是设计者设计好后, 让子类继承并实现抽象类的抽象方法

多态练习 Python

```python
# 综合案例 : 面向对象中的四大特性 (封装, 继承, 多态, 抽象)

# 导入   Abstract Base Class 抽象基类    abstractmethod 抽象方法的装饰器
from abc import ABC, abstractmethod


# Food 类
class Food(object):
    # 构造方法
    def __init__(self, name):
        self.name = name


# 子类
class Bone(Food):
    def __init__(self):
        super().__init__('骨头')


class Fish(Food):
    def __init__(self):
        super().__init__('鱼')


class Worm(Food):
    def __init__(self):
        super().__init__('虫子')


# Animal 类
class Animal(ABC):
    # 构造方法  Animal()
    def __init__(self, name):
        self.name = name

    # getter 方法
    @property
    def name(self):
        # 逻辑判断
        if self.__name:
            return self.__name      # __变量  这就是私有属性
        else:
            return '没有名字'

    # setter 方法
```

```python
    @name.setter
    def name(self, name):
        # 逻辑判断
        if isinstance(name, str):
            self.__name = name.title()
        else:
            self.__name = None      # 空对象

    # 吃的行为
    # 父类中的 eat() 行为本质上是一个抽象行为, 需要让子类必须重写该行为
    @abstractmethod
    def eat(self, food: Food):...       # 这是一个抽象方法, 没有实现了. 要求子类必须重写父类的该行为


# 子类 `继承` 父类
class Dog(Animal):
    def __init__(self, name):
        # 调用父类的构造方法
        # super(Dog, self).__init__(name)
        super().__init__(name)

    # 子类要重写父类的 eat() 行为
    def eat(self, food: Food):
        print(f'{self.name}正在吃{food.name}.')

    # 特有行为
    def bomb_blockhouse(self):
        print(f'{self.name}正在疯狂的炸碉堡...')


class Cat(Animal):
    def __init__(self, name):
        super().__init__(name)

    def eat(self, food: Food):
        print(f'{self.name}正在吃{food.name}.')

    # 特有行为
    def mymoe(self):
        print(f'{self.name}正在疯狂卖萌...')


class Bird(Animal):
    def __init__(self, name: str):
        super().__init__(name)

    def eat(self, food: Food):
```

```
93          print(f'{self.name}正在吃{food.name}.')
94
95
96      # 特有行为
97      def pole_dance(self):
98          print(f'{self.name}正在疯狂跳钢管舞...')
99
100
101 # 定义一个 Person 类
102 class Person(object):
103     # 构造方法
104     def __init__(self, name):
105         self.name = name
106
107     # 喂动物吃东西的行为
    def feed(self, animal: Animal, food: Food):     # 多态: 父类引用可以接收子类对象
108         print(f'{self.name}正在喂{animal.name}吃东西...')
109
110         # 使用 animal 对象调用 eat() 行为
111         animal.eat(food)
112
113         print(f'{animal.name}吃饱了. 准备给 {self.name} 表演一个!!!')
114
115         # 准备调用 animal 对象的特有行为
116         if isinstance(animal, Dog):
117             animal.bomb_blockhouse()
118         elif isinstance(animal, Cat):
119             animal.mymoe()
120         elif isinstance(animal, Bird):
121             animal.pole_dance()
122
123
124 # 测试
125 p = Person('孙悟空')
126
127 dog = Dog('哮天犬')
128 cat = Cat('加菲猫')
129 bird = Bird('鹦鹉')
130
131 bone = Bone()
132 fish = Fish()
133 worm = Worm()
134
135 p.feed(cat, fish)
```

输出结果 Python

```
孙悟空正在喂加菲猫吃东西...
加菲猫正在吃鱼.
加菲猫吃饱了. 准备给 孙悟空 表演一个!!!
加菲猫正在疯狂卖萌...
```

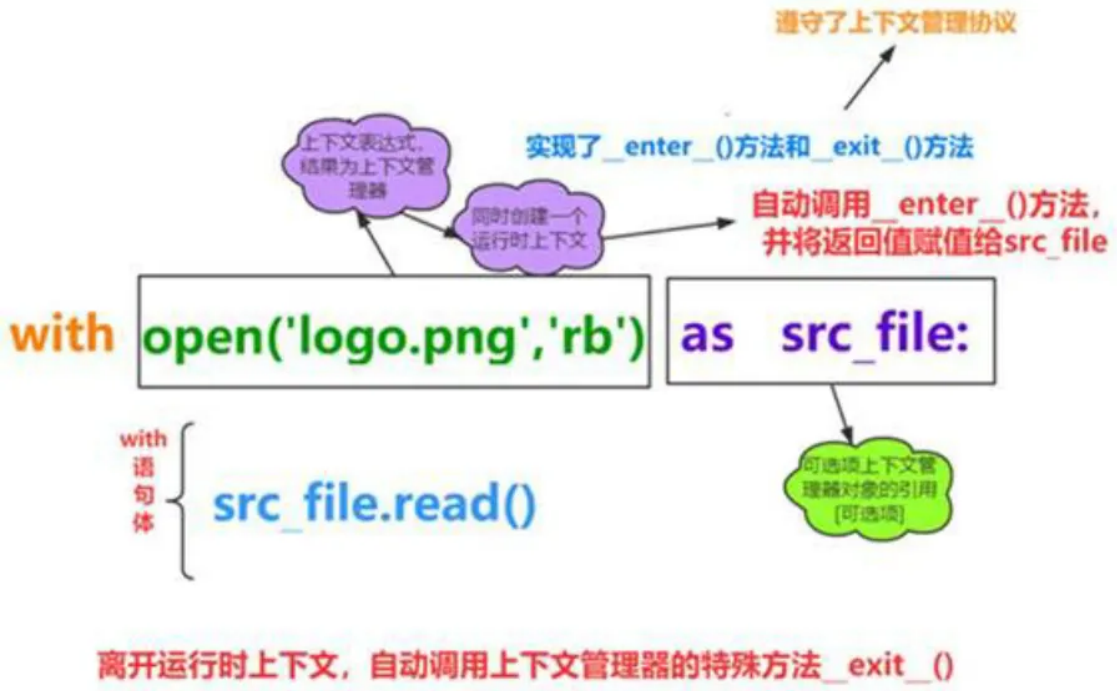
8. 文件操作

文件的类型：按文件中数据的组织形式, 文件分为以下两大类

1. 文本文件: 存储的是普通字符文件, 默认为 unicode字符集, 可以使用记事本程序打开.
2. 二进制文件: 把数据内容用 `字节` 进行存储, 无法用记事本打开, 必须使用专门的软件打开.

举例: mp3音频文件, jpg图片, doc文档等.

with语句可以自动管理上下文资源, 不论什么原因跳出with块,都能确保文件正确的关闭, 以此来达到释放资源的目的.



文本文件复制

文本文件复制 Python

```python
# 文本文件路径
source_path = r'D:\test.txt'
destination_path = r'D:\copy.txt'

# 文件文件复制
# mode='rt'  read text      mode='wt'  write text
with open(source_path, mode='rt', encoding='utf-8') as reader:
    content = reader.read()       # 读取文件中的所有数据
    with open(destination_path, mode='wt', encoding='utf-8') as writer:
        writer.write(content)
```

非文件文件复制

非文件文件复制 Python

```python
# 非文本文件路径
source_path = r'D:\桌面logo.jpg'
destination_path = r'D:\logo.jpg'

# 非文件文件复制
# mode='rb'  read binary      mode='wb'  write binary
with open(source_path, mode='rb') as reader:
    content = reader.read()       # 读取文件中的所有数据
    with open(destination_path, mode='wb') as writer:
        writer.write(content)
```

登录日志综合案例

1. 实现登录验证, 如果用户名是列表中的元素 ['Smith', 'Tom', 'Jack'], 密码 '888' 则登录成功, 否则失败
2. 不管登录是否成功, 都需要在文件中记录登录的信息
3. 登录成功, 可以看到相应的操作菜单提示, 请实现相应的功能

```python
import time

# 登录日志综合案例
FILENAME = 'log.txt'        # 相对路径


# 写入数据到日志文件中
def record_log(user, status):
    # append 拼接模式
    with open(FILENAME, mode='a+', encoding='utf-8') as f:
        chinese_time = time.strftime('%Y-%m-%d %H:%M:%S', time.localtime(time.time()))
        log = f'登录用户: {user}, {status}, 登录时间: {chinese_time}'
        f.write(log + '\n')


def read_log():
    with open(FILENAME, mode='rt', encoding='utf-8') as f:
        for line in f.readlines():
            # strip() 功能 : 去除字符串对象收尾的空格和换行
            line = line.strip()
            print(line)


# 展示界面
def show_menu():
    menu = '''
        **********************
            1. 当前登录用户
            2. 查看登录日志
            3. 退出系统
    '''
    print(menu)


# 接收用户的输入
user = input('亲, 请输入你的用户名: ')
pwd = input('亲, 请输入您的登录密码: ')

# 判断
if user in ('Jack', 'Rose', 'Peter') and pwd == '888':
    # 当前用户是注册用户, 可以登录
    record_log(user, '登录成功')
    # 保证界面不退出
    while True:
```

```
            # 展示界面
            show_menu()
            # 提供用户输入选择
            choice = input('亲, 请输入您的操作码: ')
            if choice == '1':
                print('当前用户: ', user)
            elif choice == '2':
                read_log()
            elif choice == '3':
                record_log(user, '退出系统')
                print(f'感谢您的使用, 期待下次再见!')
                break
            else:
                print('您的输入有误, 请重新选择!!!')
    else:
        # 登录失败
        print('登录失败! 您的用户名或密码错误.')
        record_log(user, '登录失败')
```

9. 异常处理

有时也可以将程序错误 (error) 称作程序异常 (exception)

相信每一位写程序的人一定会常常碰上程序错误, 过去碰上这类情况程序将终止执行, 同时出现错误信息, 错误信息内容通常是显示Traceback, 然后列出异常报告

Python可以让我们捕捉异常和撰写异常处理程序, 当异常被捕捉时会去执行异常处理程序, 然后程序可以继续执行.

9.1. 一个除数为 0 的错误

一个除数为 0 的错误 Python

```python
# 异常 : 如果不处理异常, 程序就会提前终止
def division(x, y):
    return x / y


print(division(2, 3))

# 这条语句会发生程序的异常.
# ZeroDivisionError: division by zero
print(division(4, 0))

# 分析 : 以下的输出语言可以被执行吗?
print(division(4, 1))
```

9.2. 撰写异常处理程序 try – except – else – finally

Python 的关键词 finally 功能是和 try 配合使用, 在try 之后可以有 except 或 else, 这个 finally 关键词必须放在 except 和 else 之后, 同时不论是否有异常发生一定会执行这个 finally 内的程序代码.

```python
# 异常 : 如果不处理异常, 程序就会提前终止
def division(x, y):
    # 需要将可能发生异常的逻辑代码放到 try 语句中
    try:
        # 这里有很多的逻辑执行语句...
        v = x / y
    except ZeroDivisionError as e:
        print(e)
        return '除数不能为0!'
    except TypeError as e:
        print(e)
        return '数据类型错误, 需要使用整型数据.'
    except BaseException as e:      # 兜底处理
        print(e)
        return '发生了异常!'
    else:
        # 如果 try 语句中执行一切正常, 则 else 语句会被执行.
        # 如果发生了异常, else 语句就不会被执行
        return v
    finally:
        # 这里的代码是用于释放资源 (文件资源, 连接池资源, 数据库资源...)
        # 无论是否发生了异常, finally 语句快都会在 return 之前被执行
        print('finally 语句块被执行...')


print(division(2, 3))
print(division(4, 0))
print(division(4, '2'))
print(division(4, 1))
```

9.3. 自定义异常与丢出异常

前面介绍的异常皆是Python直译器发现异常时, 自行丢出异常对象, 如果我们不处理程序就终止执行, 如果我们使用 try–except 处理程序可以在异常中继续执行.

我们自己将它定义为异常然后丢出异常信息, 程序停止正常往下执行, 同时让程序跳到自己设计的 except 去执行.

```python
# 自定义异常类
class AgeException(BaseException):
    # 构造方法
    def __init__(self, msg):
        super().__init__(msg)

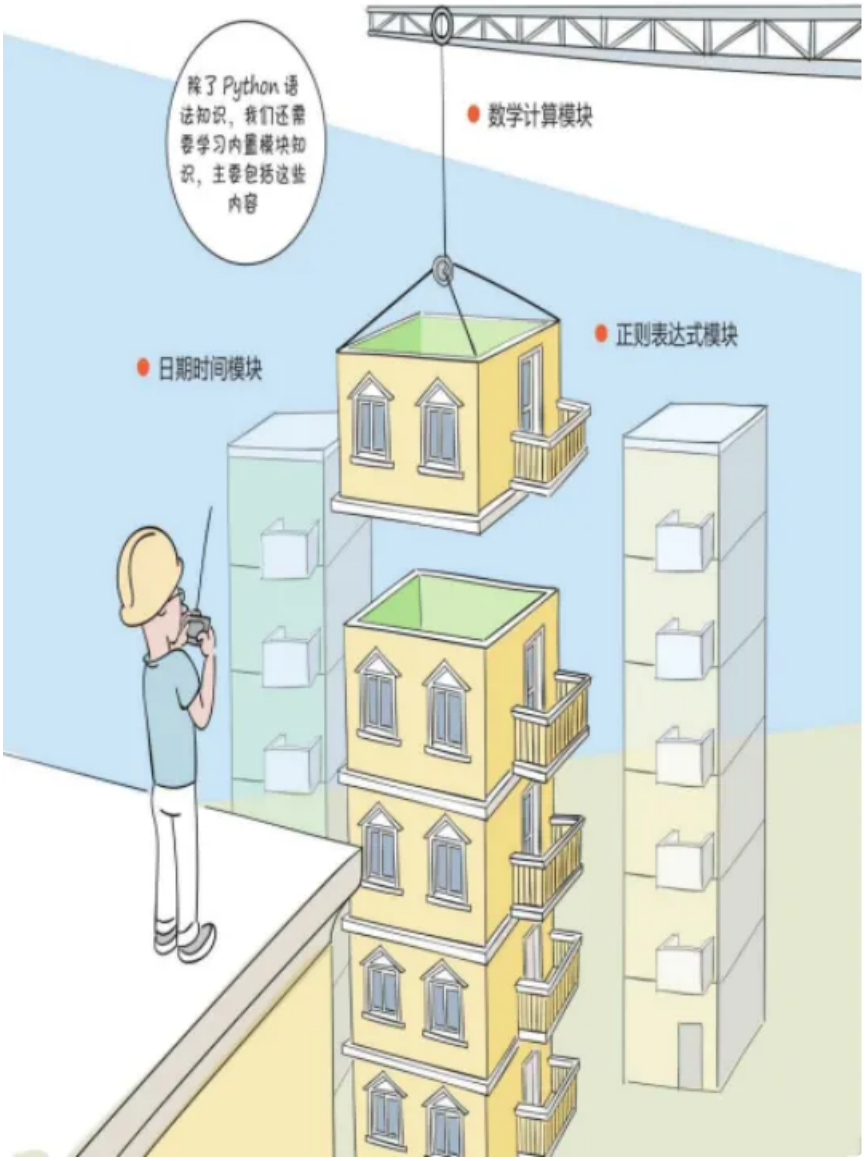

# 自定义函数
def check_age(age):
    if age < 18:
        # 抛出自定义异常对象
        raise AgeException('你未成年, 不能购买烟酒, 不能去98嗨~~')
    else:
        print('恭喜你成年了, 来, 跟我一起high~~~')


# 测试
try:
    age = int(input('亲, 请输入您的年龄: '))
    # 调用自定义函数
    check_age(age)
except AgeException as e:
    print(e)


print('继续执行后续代码...')
```

10. 模块化编程

在真正做项目时, 我们会使用别人已经开发好的模块, 这样就不必从零开发项目了, 还可以加快开发速度, 这些模块可能是Python官方提供的, 也可能是第三方开发的.Python官方提供的模块,就叫做 内置模块.



Python可以说是一个模块化编程的高级语言.

10.1. math 模块

在math模块中包含数学运算相关的函数, 例如指数, 对数, 平方根和三角函数等.

Python

```python
# 导入模块
import math

# 20实际最美的数学公式 : 欧拉 e的iπ + 1 = 0

# math 模块
print(math.pi)
print(math.tau)
print(math.e)

# 阶乘 (递归函数)
r = math.factorial(5)
print(r)

# 欧几里得 (初高中几何 -> 源自于 <几何原本>)
# 最大公约数
print(math.gcd(40, 104))

# 最小公倍数
print(math.lcm(40, 104))
```

输出结果 Python

```
3.141592653589793
6.283185307179586
2.718281828459045
120
8
520
```

10.2. random 模块

所谓的随机数是指平均散布在某区间的数字, 随机数其实用途很广, 最常见的应用是设计游戏时可以控制输出结果, 赌场的老虎机器就是靠它赚钱. 我们将介绍random模块中最有用的几个方法, 同时也会分析赌场赚钱的原因.

函数名称	说明
randint(x, y)	产生 x(含) 到 y(含) 的随机整数
random( )	产生 0(含) 到 1(不含) 的随机浮点数
uniform(x, y)	产生 x(含) 到 y(不含) 的随机浮点数
choice(列表)	可以在列表中随机回传一个元素
shuffle(列表)	将列表元素重新排列
sample(列表 , 数量)	随机回传第 2 个参数数量的列表元素
seed(x)	X 是种子值，未来每次可以产生相同的随机数

Python

```python
import random


# 执行范围的随机整数
r1 = random.randint(1, 10)
print(f'{r1 = }')

# 0.0 ~ 1.0 浮点值
print(random.random())
print(random.choice(['a', 'b', 'c']))
print(random.sample(['a', 'b', 'c'], 2))

alist = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
random.shuffle(alist)       # 洗牌, 随机置换
print(alist)
```

10.3. 日期时间模块

strftime( ) 参数	意义
%Y	含世纪的年份，例如：‘2020’
%y	不含世纪的年份，例如：‘20’ 代表 2020
%B	用完整英文代表月份，例如：‘January’ 代表 1 月
%b	用缩写英文代表月份，例如：‘Jan’ 代表 1 月
%m	用数字代表月份，‘01’ ～ ‘12’
%j	该年的第几天，‘001’ ～ ‘366’
%d	该月的第几天，‘01’ ～ ‘31’
%A	用完整英文代表星期几，例如：‘Sunday’ 代表星期日
%a	用缩写英文代表星期几，例如：‘Sun’ 代表星期日
%w	用数字代表星期几，‘0’ 星期日～ ‘6’ 星期六
%H	24 小时制，‘00’ ～ ‘23’
%I	12 小时制，‘01’ ～ ‘12’
%M	分，‘00’ ～ ‘59’
%S	秒，‘00’ ～ ‘59’
%p	‘AM’ 或 ‘PM’

datetime 对象转成字符串 Python

```python
# 第一个 datetime 表示的是模块的名称
# 第二个 datetime 表示的是类名
from datetime import datetime, timedelta        # 微小的时间间隔

today = datetime.today()
print(f'{today = }')
now = datetime.now()
print(f'{now = }')

# 将日期时间对象按照指定的模式转换为字符串格式时间数据
# f -> format 格式化
now_str = now.strftime('%Y-%m-%d %H:%M:%S')
print(now_str, type(now_str))

# 将字符串类型的数据转换为时间日期类型的数据
# p -> parse 解析
date_str = '2008年08月08日 00:10:50'
d = datetime.strptime(date_str, '%Y年%m月%d日 %H:%M:%S')
print(f'{d = }, {type(d)}')

# timedelta 微小的时间间隔
# 需求 : 从现在向后推, 100 之后是哪一天 ?
# now + 100       # datetime + int
future = now + timedelta(days=100)
print(f'{future = }')
```

输出结果 Python

```
today = datetime.datetime(2025, 4, 11, 0, 15, 1, 965610)
now = datetime.datetime(2025, 4, 11, 0, 15, 1, 965670)
2025-04-11 00:15:01 <class 'str'>
d = datetime.datetime(2008, 8, 8, 0, 10, 50), <class 'datetime.datetime'>
future = datetime.datetime(2025, 7, 20, 0, 15, 1, 965670)
```

10.4. 自定义模块

思考：为什么要自定义模块?

答：在实际开发中, Python提供的标准库模块不能满足开发需求, 程序员需要一些个性化的模块, 就可以进行自定义模块的实现.

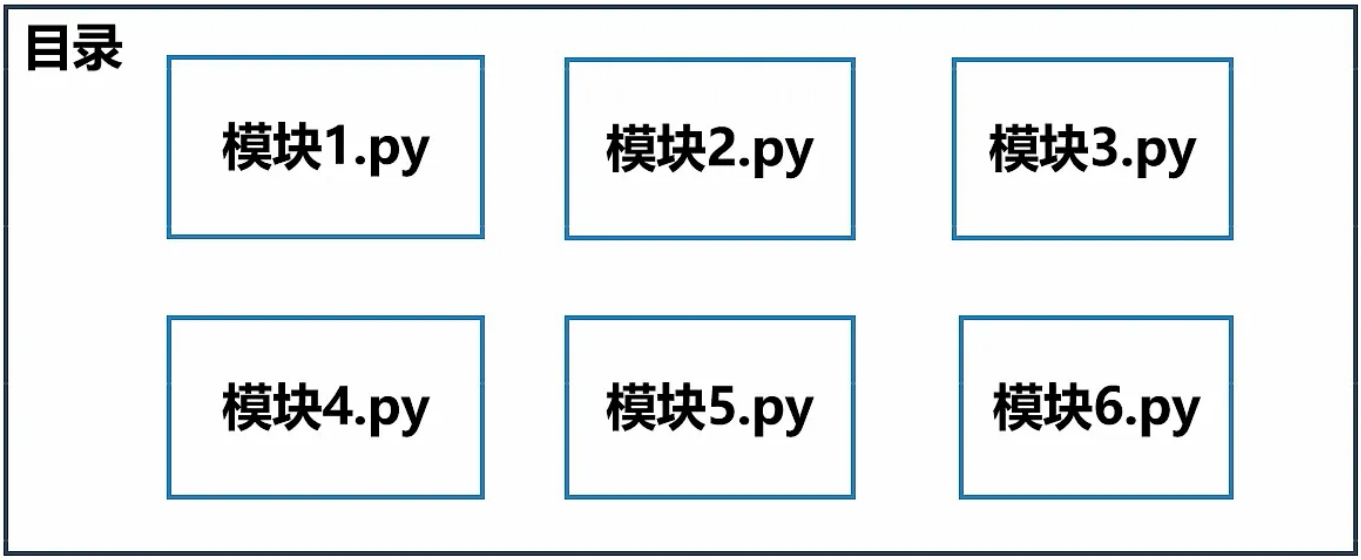
说明：模块导入 – 基本语法

[from 模块名] import (函数 | 类 | 变量 | *) [as 别名]

1. [ ] 是可选项
2. 可以根据需求, 选择合适的形式进行导入

思考：为什么需要包?

在一个实际的项目, 可能需要很多个模块, 当模块文件越来越多, 如果我们将所有的模块文件都放在同一个文件夹, 就会带来很多问题, 不利于管理和调用, 怎么办?



说明：

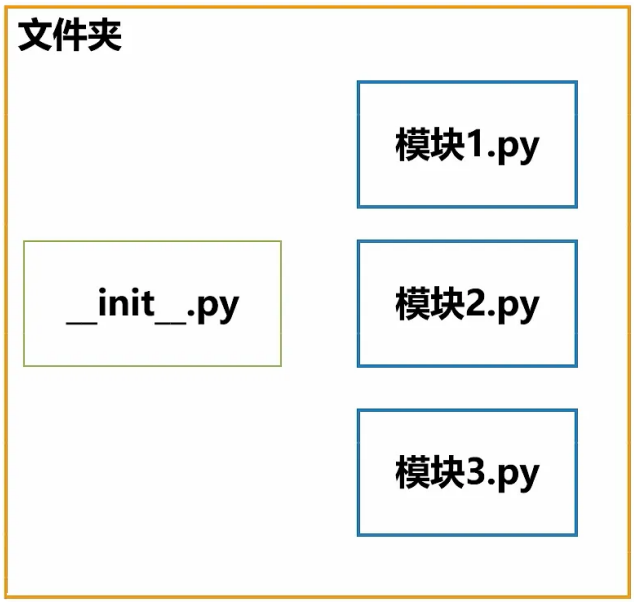
1. 这里写了六个模块文件, 实际上可能有很多的模块文件.

2. 把所有的模块文件, 都放到一个目录下, 必然造成管理的混乱.

3. 怎么办? –> 包







说明:

1、从结构上看, 包就一个文件夹, 在该文件夹下包含了一个 __init__.py 文件

2、该文件夹可以包含多个模块文件, 从逻辑上看, 包可以视为模块集合



准备工作

准备工作：makefood.py

Python

```python
# 在自定义模块中定义一些功能

# 制作冰淇淋
def make_icecream(*toppings):
    print('这个冰淇淋的配料如下: ')
    for topping in toppings:
        print('\t', topping)


# 制作饮料
def make_drink(size, drink):
    print('您点的饮料如下: ')
    print('\t', size.title())
    print('\t', drink.title())


# 模块测试
if __name__ == '__main__':
    make_icecream('草莓酱', '葡萄干', '巧克力碎片')
    make_drink('大杯', 'black coffee')
```

测试文件

Python

```python
# import makefood
# from makefood import make_icecream, make_drink
import makefood as m


# main 函数
if __name__ == '__main__':

    # 使用模块
    # makefood.make_icecream('草莓酱', '葡萄干', '巧克力碎片')
    # makefood.make_drink('大杯', 'black coffee')

    # make_icecream('草莓酱', '葡萄干', '巧克力碎片')
    # make_drink('大杯', 'black coffee')

    m.make_icecream('草莓酱', '葡萄干', '巧克力碎片')
    m.make_drink('大杯', 'black coffee')

    print('hello world!')
```

输出结果 Python

```
这个冰淇淋的配料如下:
      草莓酱
      葡萄干
      巧克力碎片
您点的饮料如下:
      大杯
      Black Coffee
hello world!
```

10.5. 第三方模块

看见更大的 Python 世界 -> https://pypi.org/

pip有哪些命令可以使用，可以使用 `pip help` 命令来查看：

```
$ pip help

Usage:
  pip <command> [options]

Commands:
  install                     Install packages.
  download                    Download packages.
  uninstall                   Uninstall packages.
  freeze                      Output installed packages in requirements format.
  list                        List installed packages.
  show                        Show information about installed packages.
  check                       Verify installed packages have compatible
                              dependencies.
  config                      Manage local and global configuration.
  search                      Search PyPI for packages.
  wheel                       Build wheels from your requirements.
  hash                        Compute hashes of package archives.
  completion                  A helper command used for command completion.
  help                        Show help for commands.
```

如上图有：

pip install 安装软件包

pip download 只下载软件包不安装

pip uninstall 卸载软件包

pip list 显示已安装了哪些软件包

pip search 在pypi上模糊搜索软件包等等...

用pip安装第三方包

上面已经举例子了，如果你知道具体软件包的名字，就直接使用 `pip install 软件包名` 进行安装，这样会安装该软件的最新版本。

同时pip会计算该软件的依赖包，如果没有冲突，就会一并把该软件包的依赖包一起安装了。

思考：什么是依赖?

就是一个软件要依赖另一个软件的功能才能运行，必须要把相关的依赖包都安装了，才能愉快的使用。

我们拿安装requests库来举例。

```
$ pip install requests

Looking in indexes: https://pypi.org/simple
Collecting requests
  Using cached
  https://files.pythonhosted.org/packages/7d/e3/
  20f3d364d6c8e5d2353c72a67778eb189176f08e873c9900e10c0287b84b/
  requests-2.21.0-py2.py3-none-any.whl
Collecting chardet<3.1.0,>=3.0.2 (from requests)
  Using cached https://files.pythonhosted.org/packages/bc/a9/
  01ffebfb562e4274b6487b4bb1ddec7ca55ec7510b22e4c51f14098443b8/
  chardet-3.0.4-py2.py3-none-any.whl
Collecting idna<2.9,>=2.5 (from requests)
  Using cached https://files.pythonhosted.org/packages/14/2c/
  cd551d81dbe15200be1cf41cd03869a46fe7226e7450af7a6545bfc474c9/
  idna-2.8-py2.py3-none-any.whl
Collecting urllib3<1.25,>=1.21.1 (from requests)
  Using cached https://files.pythonhosted.org/packages/62/00/
  ee1d7de624db8ba7090d1226aebefab96a2c71cd5cfa7629d6ad3f61b79e/
  urllib3-1.24.1-py2.py3-none-any.whl
Collecting certifi>=2017.4.17 (from requests)
  Using cached https://files.pythonhosted.org/packages/9f/e0/
  accfc1b56b57e9750eba272e24c4dddeac86852c2bebd1236674d7887e8a/
  certifi-2018.11.29-py2.py3-none-any.whl
Installing collected packages: chardet, idna, urllib3, certifi, requests
```

安装requests时，会向pypi上去查找这个包，然后计算和安装依赖，可以看出requests还依赖chardet、idna、urllib3这些包，所以会一并安装。

pip镜像源安装软件包

在用 pip install 安装时，是直接下载 pypi 上的软件，各种原因我们访问国外网站有时比较慢，可能在安装时会很慢，甚至提示超时，安装失败。

所以国内有公司和大学就镜像了pypi，把上面的软件包都镜像到国内，通过他们的镜像源安装就会很快。

比较典型的镜像源有：

清华：https://pypi.tuna.tsinghua.edu.cn/simple/

阿里云：http://mirrors.aliyun.com/pypi/simple/

中国科技大学：https://mirrors.ustc.edu.cn/pypi/simple/

华中科技大学：https://pypi.hustunique.com/simple/

上海交通大学：https://mirror.sjtu.edu.cn/pypi/web/simple/

豆瓣：http://pypi.douban.com/simple

比如使用清华大学镜像源安装就是：

`pip install 软件包名 -i https://pypi.tuna.tsinghua.edu.cn/simple`

`pip install requests -i http://pypi.douban.com/simple --trusted-host pypi.douban.com`

镜像源没法实时做到更新pypi上的软件包，所以有些软件包的版本可能不是最新的。

卸载软件包

使用如下卸载命令：`pip uninstall 软件包名`

就可以卸载软件包，卸载软件包之前最好先用show命令看一下该软件包的被依赖关系，如果其它包依赖你要卸载的软件包，那么你卸载后，其它软件包就没法运行了。

`pip show 软件包名`

使用pip show命令可以查看该包的一些信息。比如查看requests库的信息：`pip show requests`

```
$ pip show requests

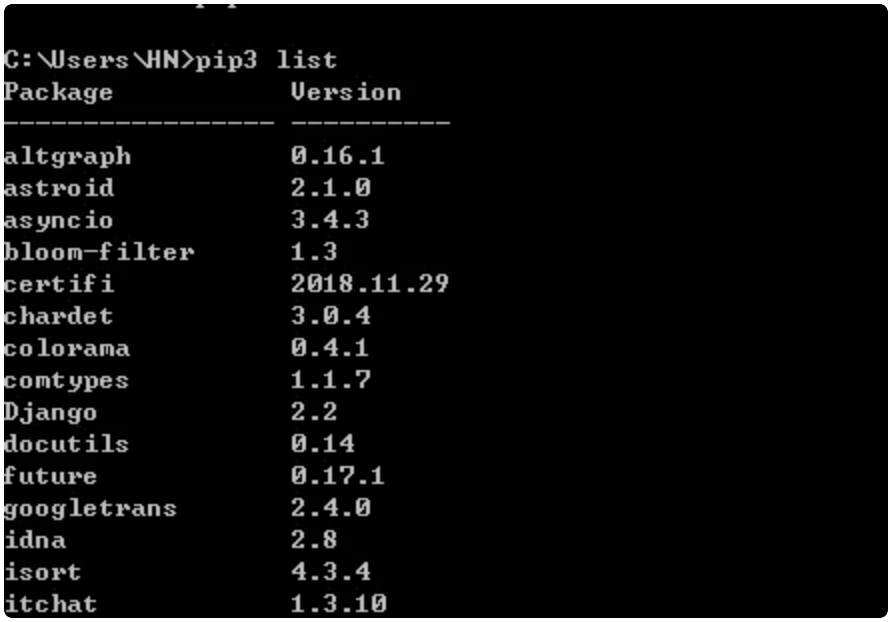
Name: requests
Version: 2.21.0
Summary: Python HTTP for Humans.
Home-page: http://python-requests.org
Author: Kenneth Reitz
Author-email: me@kennethreitz.org
License: Apache 2.0
Location: py37\lib\site-packages
Requires: certifi, chardet, idna, urllib3
Required-by:
```

可以查看该软件包的作者，软件安装路径，依赖的库和被依赖的库等息息。

要注意看Required-by那一项，如果冒号后面显示有其他的库，那你卸载requests库就要小心，因为它被其他库依赖了，卸载的话其他库就会出问题。

显示本地软件包

如果我们卸载软件包，忘记该软件包名的全称了呢，可以使用list命令：`pip list`



list命令显示本地所有安装的第三方库和相应的版本号，这时就可以查看完整的软件包名是怎么写的。

OK，pip入门指南说完了，当然python的包管理工具也不只有pip。

还有Conda、Pipenv、Poetry等，哪个用得熟练就用那个。

python常用第三方库总结

Python语言有近60万个第三方库，覆盖信息技术几乎所有领域。下面简单介绍下网络爬虫、自动化、数据分析与可视化、WEB开发、机器学习和其他常用的一些第三方库，如果有你感兴趣的库，不妨去试试它的功能吧。

10.5.1. 网络爬虫

- **requests**–对HTTP协议进行高度封装，支持非常丰富的链接访问功能。
- **PySpider**–一个国人编写的强大的网络爬虫系统并带有强大的WebUI。
- **bs4**–beautifulsoup4库，用于解析和处理HTML和XML。

- **Scrapy**–很强大的爬虫框架，用于抓取网站并从其页面中提取结构化数据。可用于从数据挖掘到监控和自动化测试的各种用途
- **Crawley**–高速爬取对应网站的内容，支持关系和非关系数据库，数据可以导出为JSON、XML等
- **Portia**–可视化爬取网页内容
- **cola**–分布式爬虫框架
- **newspaper**–提取新闻、文章以及内容分析
- **lxml**–lxml是python的一个解析库，这个库支持HTML和xml的解析，支持XPath的解析方式

10.5.2. 自动化

- **XlsxWriter**–操作Excel工作表的文字，数字，公式，图表等
- **win32com**–有关Windows系统操作、Office（Word、Excel等）文件读写等的综合应用库
- **pymysql**–操作MySQL数据库
- **pymongo**–把数据写入MongoDB
- **smtplib**–发送电子邮件模块
- **selenium**–一个调用浏览器的driver，通过这个库可以直接调用浏览器完成某些操作，比如输入验证码，常用来进行浏览器的自动化工作。
- **pdfminer**–一个可以从PDF文档中提取各类信息的第三方库。与其他PDF相关的工具不同，它能够完全获取并分析 P D F 的文本数据
- **PyPDF2**–一个能够分割、合并和转换PDF页面的库。
- **openpyxl**– 一个处理Microsoft Excel文档的Python第三方库，它支持读写Excel的xls、xlsx、xlsm、xltx、xltm。
- **python-docx**–一个处理Microsoft Word文档的Python第三方库，它支持读取、查询以及修改doc、docx等格式文件，并能够对Word常见样式进行编程设置。

10.5.3. 数据分析及可视化

- **matplotlib**–Matplotlib 是一个 Python 2D 绘图库，可以生成各种可用于出版品质的硬拷贝格式和跨平台交互式环境数据。Matplotlib 可用于 Python 脚本，Python 和 IPython shell（例如 MATLAB 或 Mathematica），Web 应用程序服务器和各种图形用户界面工具包。”
- **numpy**–NumPy 是使用 Python 进行科学计算所需的基础包。用来存储和处理大型矩阵，如矩阵运算、矢量处理、N维数据变换等。
- **pyecharts**–用于生成 Echarts 图表的类库
- **pandas**–一个强大的分析结构化数据的工具集，基于numpy扩展而来，提供了一批标准的数据模型

和大量便捷处理数据的函数和方法。

- **Scipy**: 基于Python的matlab实现，旨在实现matlab的所有功能，在numpy库的基础上增加了众多的数学、科学以及工程计算中常用的库函数。
- **Plotly**–Plotly提供的图形库可以进行在线WEB交互，并提供具有出版品质的图形，支持线图、散点图、区域图、条形图、误差条、框图、直方图、热图、子图、多轴、极坐标图、气泡图、玫瑰图、热力图、漏斗图等众多图形
- **wordcloud**–词云生成器
- **jieba**–中文分词模块

10.5.4. WEB开发

- **Django**–一个开放源代码的Web应用框架，由Python写成。 是Python生态中**最流行的开源**Web应用框架，Django采用模型、模板和视图的编写模式，称为MTV模式。
- **Pyramid**是一个通用、开源的Python Web应用程序开发框架。它主要的目的是让Python开发者更简单的创建Web应用，相比Django，Pyramid是一个**相对小巧、快速、灵活**的开源Python Web框架。
- **Tornado**–一种 Web 服务器软件的开源版本。Tornado和现在的主流Web服务器框架（包括大多数Python的框架）有着明显的区别：它是非阻塞式服务器，而且速度相当快
- **Flask**是轻量级Web应用框架，相比Django和Pyramid，它也被称为**微框架**。使用Flask开发Web应用十分方便，甚至几行代码即可建立一个小型网站。Flask核心十分简单，并不直接包含诸如数据库访问等的抽象访问层，而是通过扩展模块形式来支持。

10.5.5. 机器学习

- **NLTK**–一个自然语言处理的第三方库，NLP领域中常用，可建立词袋模型（单词计数），支持词频分析（单词出现次数）、模式识别、关联分析、情感分析（词频分析+度量指标）、可视化（+matploylib做分析图）等
- **TensorFlow**–谷歌的第二代机器学习系统，是一个使用数据流图进行数值计算的开源软件库。
- **Keras** –是一个高级神经网络 API，用 Python 编写，能够在 TensorFlow，CNTK 或 Theano 之上运行。它旨在实现快速实验，能够以最小的延迟把想法变成结果，这是进行研究的关键。”
- **Caffe**–一个深度学习框架，主要用于计算机视觉，它对图像识别的分类具有很好的应用效果
- **theano**–深度学习库。它与Numpy紧密集成，支持GPU计算、单元测试和自我验证，为执行深度学习中大规模神经网络算法的运算而设计，擅长处理多维数组。
- **Scikit-learn**–是一个简单且高效的**数据挖掘和数据分析工具**，它基于NumPy、SciPy和matplotlib构建。Scikit-learn的基本功能主要包括6个部分：分类，回归，聚类，数据降维，模型选择和数据预处理。Scikit-learn也被称为sklearn。

10.5.6. 其他常用

- **IPython**–一个基于Python 的交互式shell，比默认的Python shell 好用得多，支持变量自动补全、自动缩进、交互式帮助、魔法命令、系统命令等，内置了许多很有用的功能和函数
- **PTVS**–Visual Studio 的 Python 工具
- **pydub**–支持多种格式声音文件，可进行多种信号处理、信号生成、音效注册、静音处理等
- **TimeSide**–能够进行音频分析、成像、转码、流媒体和标签处理的Python框架
- **dnspython**–DNS工具包
- **pygame**–专为电子游戏设计的一个模块
- **PyQt5**–pyqt5是Qt5应用框架的Python第三方库，编写Python脚本的应用界面
- **PIL(Pillow)**–PIL库是Python语言在**图像处理方面**的重要第三方库，支持图像存储、显示和处理，它能够处理几乎所有图片格式，可以完成对图像的缩放、剪裁、叠加以及向图像添加线条、图像和文字等操作
- **OpenCV**–图像和视频工作库
- **Py2exe**: 将python脚本转换为windows上可以独立运行的可执行程序。
- **WeRoBot** 是一个**微信公众号开发框架**，也称为的微信机器人框架。WeRoBot可以解析微信服务器发来的消息，并将消息转换成成Message或者Event类型。